

svy: A Python package for end-to-end design and analysis of complex survey data

Mamadou S. Diallo ^{1¶}

¹ Samplix, United States ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 22 July 2026

Published: unpublished

License

Authors of papers retain copyright[¶] and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/))[¶]

Summary

Probability-based surveys are one of the main tools used by governments, international organizations, and researchers to produce statistics that guide decision-making. For example, national household surveys inform policy on poverty, health, education, and employment for hundreds of millions of people each year. Similarly, opinion polling and market research surveys inform corporations, media, and other organizations on populations' views and preferences, and social science researchers rely on complex samples to study behaviors and living conditions. To be efficient and operationally feasible, these surveys rely on stratification, multi-stage cluster sampling, unequal probabilities of selection, and post-collection weight adjustments. As a result, standard statistical methods, which assume independent and identically distributed observations, produce incorrect variance estimates and invalid confidence intervals when applied directly to survey data (Kish, 1965; Lohr, 2021). Design-based inference, which uses the known sampling mechanism, is the standard framework for analyzing such data.

svy is an open-source Python package for the design and analysis of complex survey samples. It covers the full survey workflow: sample size calculation, sample selection (simple random, systematic, and probability proportional to size methods, including multi-stage designs), weight adjustment (nonresponse adjustment, poststratification, GREG calibration, raking, and trimming), estimation of means, totals, proportions, ratios, and medians with Taylor linearization or replication-based variance (BRR, Fay-BRR, jackknife, bootstrap, and SDR), design-adjusted categorical analysis (tabulation, t-tests, rank tests, and Rao-Scott tests), and generalized linear models with design-adjusted standard errors. The package is built on Polars (Polars contributors, 2025), NumPy (Harris et al., 2020), and SciPy (Virtanen et al., 2020), and its performance-critical routines are implemented in Rust.

svy is the successor of samplix (Diallo, 2021, 2022), an earlier Python package for complex survey analysis by the same author. samplix has been retired, and svy, a ground-up redesign informed by several years of samplix usage, is its replacement and the recommended migration target.

Statement of need

Software support for design-based inference is mature in several environments: the survey package for R (Lumley, 2004, 2010), the survey procedures in SAS, the svy prefix in Stata, the SPSS Complex Samples module, and specialized tools such as SUDAAN. However, Python, despite being a leading language for data science and increasingly the primary language of analysts at national statistical offices, public health programs, and research organizations, has lacked a comprehensive and production-oriented equivalent. In practice, analysts working in Python had to move their data to R or Stata for design-based inference, or apply weighted but design-naïve methods that understate uncertainty.

samplics (Diallo, 2021) was the first Python package to offer a broad set of design-based methods, and it demonstrated the demand for survey methodology tools in the Python ecosystem. Experience with samplics in production settings also revealed structural limitations that could not be addressed incrementally. For example, design information had to be passed manually to each function, weight adjustments were disconnected from the replicate weights, and performance was a constraint on large surveys. svy is a ground-up redesign built around five goals:

1. Coverage of the full survey lifecycle. A single coherent API spans planning (sample size calculation for estimation and comparison objectives), selection (including multi-stage designs where inclusion probabilities are chained across stages automatically), weighting, estimation, testing, and modeling.
2. Correctness by construction. The central `Sample` object binds the data to a `Design` object (strata, sampling units, weights, and replicate weights) once, validates the combination, and is immutable; every transformation returns a new `Sample` with the design metadata propagated automatically. Furthermore, subpopulation analysis via the `where` argument retains the full design structure for variance estimation, matching the behavior of `subset()` on a survey design object in R. This matters because filtering the rows before analysis, a common practice, generally understates the standard errors.
3. Replicate-weight fidelity. When replicate weights are created from the design weights, every subsequent adjustment (nonresponse, poststratification, calibration, raking, and trimming) is applied to the replicate weights in the same pass as the full-sample weights. Hence, replication-based standard errors reflect the variability of the entire weighting process, not just sampling variability, without additional effort from the analyst (Valliant & Dever, 2018; Wolter, 2007).
4. Performance. Variance estimation and other computational bottlenecks are implemented in Rust and distributed as the companion wheel `svy-rs`, while data operations use Polars. A full pipeline consisting of selection, weighting with 500 bootstrap replicates, and estimation runs in under a quarter of a second on a standard laptop. This makes the package practical for national-scale surveys and for simulation studies.
5. Reproducibility and automation. An `svy` analysis is expressed entirely in code: data transformations are performed through the built-in wrangling namespace rather than ad hoc outside the package, random operations accept explicit seeds, and the `Sample` object manages its metadata internally, i.e., the design roles, variable and category labels, and the full weight history travel with the data instead of living in external documentation. Every analytical method returns a typed result object (e.g., `Estimate`, `Table`, `TTest`, and the GLM fit objects), and the `serialize` module converts these results to stable, versioned, machine-readable structures. Hence, `svy` pipelines can be rerun, audited, and integrated into automated production systems, and the typed, validated interfaces provide a reliable contract for emerging AI-assisted workflows, where generated analysis code must be checked and reproduced rather than trusted.

The primary target audiences are survey statisticians, national statistical offices, public health and development programs, opinion polling and market research organizations, and social science researchers and data scientists working with complex samples from large survey programs such as ACS and NHANES in the United States, EU-SILC in Europe, and DHS and LSMS internationally.

State of the field

The survey package for R (Lumley, 2004) is the reference open-source implementation of design-based inference, and it served as the primary benchmark for `svy`. Across design specification, Taylor linearization, replication methods, calibration, categorical analysis, and

90 generalized linear models, `svy` and `survey` produce numerically identical results to at least
91 six significant digits. The few intentional differences are documented and align with the
92 conventions used in `Stata`. The full side-by-side comparison, with reproducible R and Python
93 code, is available at <https://svylab.com/learn/notes/posts/svy-vs-r-comparison/>.

94 Within Python, `samplics` (Diallo, 2021, 2022) was previously the most comprehensive option;
95 it is now retired in favor of `svy`. Other Python tools address individual pieces of the workflow,
96 e.g., weighting adjustments or weighted descriptive statistics, but not design-based variance
97 estimation across the full analysis lifecycle. Relative to the R `survey` package itself, `svy` also
98 integrates the pre-collection stages (sample size calculation and multi-stage sample selection)
99 into the same object model used for analysis. Hence, a survey can be planned, drawn, weighted,
100 and analyzed without leaving the package.

101 The statistical methodology follows the established literature: calibration and model-assisted
102 estimation (Deville & Särndal, 1992; Särndal et al., 1992), replication variance estimation
103 (Wolter, 2007), Rao-Scott corrections for categorical tests (Rao & Scott, 1981, 1984), and
104 design-adjusted GLMs via linearization of the estimating equations (Binder, 1983).

105 Software design

106 `svy` exposes two primary objects. `Design` is a lightweight, frozen descriptor that maps dataset
107 columns to their roles in the sampling design, i.e., strata, primary and secondary sampling
108 units, weights, selection probabilities, measures of size, and the replicate weight configuration.
109 `Sample` binds a Polars `DataFrame` to a `Design`, validates the combination at construction,
110 and organizes the functionality into focused namespaces: `sampling` for selection, weighting,
111 estimation, categorical, `glm`, and `wrangling` for design-aware data transformations. The
112 `wrangling` methods keep the design, weights, and replicate weights synchronized; for example,
113 when a design column is renamed, the `Design` object is updated automatically.

114 Both objects are immutable, so analytical pipelines are auditable and reproducible by
115 construction. Each weighting step adds a named weight column and preserves the full weight
116 history, and there is no risk of silently overwriting the design or losing track of the active
117 weights. Every estimation call returns a typed result object that renders as a formatted table
118 and can be exported to a Polars `DataFrame`.

119 The following example gives a taste of the API. It loads one of the bundled datasets, binds
120 the data to its sampling design, creates a literacy indicator and a simulated response status,
121 adjusts the weights for nonresponse, trims extreme weights, and estimates the adult literacy
122 rate by urban and rural areas with design-based standard errors. Each step returns a new
123 `Sample` object, hence the chained style:

```
import svy
import numpy as np

rng = np.random.default_rng(147)

# A bundled example survey (runs offline)
data = svy.datasets.load("ind_sample_wb_2023", source="bundled")
info = svy.datasets.describe("ind_sample_wb_2023", source="bundled")

# Bind the data to its sampling design
sample = svy.Sample(data, svy.Design(**info.design))

literacy_rate = (
    sample
    # 1. Wrangle: a 1/0 literacy indicator, plus a response status
    .wrangling.mutate({
```

```

"literate": svy.when(svy.col("literacy") == "Yes").then(1)
               .when(svy.col("literacy") == "No").then(0)
               .otherwise(None),
# simulated for illustration; the example data has 100% response
"resp_status": rng.choice(
    ["respondent", "non-respondent"], p=[0.85, 0.15],
    size=sample.n_records,
),
})
# 2. Adjust the weights for nonresponse, within urban/rural classes
.weighting.adjust(
    resp_status="resp_status",
    by="urbrur",
    resp_mapping={"rr": "respondent", "nr": "non-respondent"},
    wgt_name="nr_wgt",
)
# 3. Trim extreme weights to reduce variance
.weighting.trim(upper=3.0)
# 4. Adult (15+) literacy rate by area, with design-based SEs
.estimation.mean(
    "literate",
    by="urbrur",
    where=svy.col("age") >= 15,
    drop_nulls=True,
)
)

```

```
print(literacy_rate)
```

```
124 Estimate: MEAN (TAYLOR)
```

```
125 where: age >= 15
```

```
126
```

127	urbrur	est	se	lci	uci	cv (%)
128	Rural	0.7516	0.0466	0.6558	0.8474	6.20
129	Urban	0.9183	0.0188	0.8797	0.9570	2.05

130 The result reports, for each domain, the point estimate, its standard error, the 95% confidence
 131 interval, and the coefficient of variation, all computed under the survey design that was declared
 132 once at the start of the pipeline. The by argument produces domain estimates with standard
 133 errors that follow the sample domain estimation theory, and the where argument restricts the
 134 estimation to a subpopulation while keeping the full sample for variance estimation, rather
 135 than filtering the rows.

136 The package is distributed on PyPI (`pip install svy`) under the MIT license, with the Rust
 137 engine `svy-rs` shipped as a separate wheel. `svy` is also the core of a growing ecosystem of
 138 survey packages that share its design conventions: `svy-io` reads and writes SPSS, Stata,
 139 and SAS files, and `svy-sae` provides small area estimation. Additional extensions are under
 140 development. For documentation and teaching, `svy` bundles an offline-complete synthetic
 141 country derived from the World Bank synthetic census and survey data (Solatorio & Dupriez,
 142 2023; World Bank, 2023a, 2023b). Hence, every example in the documentation is reproducible
 143 without network access.

144 Research impact statement

145 `svy` inherits and extends the user base of `samplics`, which has been used since 2020 in applied
 146 survey work and research across public health, official statistics, and international development

settings. `samplics` has been downloaded more than 278,000 times from PyPI, including an estimated 33,000 direct user installs after excluding known mirrors and continuous integration systems, and its usage has kept growing through 2026. `svy` was first released on PyPI in July 2025 and has been downloaded nearly 50,000 times in its first thirteen months, with direct user installs increasing month over month and exceeding 1,500 in July 2026 alone. With the retirement of `samplics`, `svy` is the maintained, comprehensive package for design-based survey inference in Python. The published numerical validation against the R survey package provides the correctness evidence that organizations require before adopting a new tool for the production of official statistics. The ambition, first stated for `samplics` (Diallo, 2022), remains the same: a robust, comprehensive, and easy to use Python ecosystem for survey sampling and the production of official statistics, with `svy` at its core. Documentation, tutorials, and methodological notes are maintained at <https://svylab.com/docs/svy>.

AI usage disclosure

`samplics` was developed before the mainstream use of generative AI for software development, and none of its code was written by AI. The development of `svy` began by porting the `samplics` implementation to the new API design described in this paper: the author fully designed the software and used the `samplics` code base to build an initial working prototype of `svy`. At that point, generative AI tools were introduced to polish the code, fix bugs, and improve performance, and they played a critical role in the development of the Rust engine `svy-rs`. All AI-assisted code was reviewed and tested by the author, and the numerical validation against the R survey package applies to the final code regardless of how it was produced.

This manuscript was prepared with the assistance of Claude Fable 5 (Anthropic), drawing on the author's previously written manuscripts and documentation, and was revised and finalized through several rounds of review and editing by the author.

Acknowledgements

The author thanks the World Bank Development Data Group for making the synthetic household survey data publicly available, and Thomas Lumley, whose survey package for R has set the standard for design-based inference in statistical software and served as the reference implementation for validating `svy`. The author is also grateful to the users of `samplics` and `svy` who reported issues, suggested features, and contributed in many ways to the development of both packages over the years.

References

- Binder, D. A. (1983). On the variances of asymptotically normal estimators from complex surveys. *International Statistical Review*, 51(3), 279–292.
- Deville, J.-C., & Särndal, C.-E. (1992). Calibration estimators in survey sampling. *Journal of the American Statistical Association*, 87(418), 376–382.
- Diallo, M. S. (2021). `Samplics`: A Python package for selecting, weighting and analyzing data from complex sampling designs. *Journal of Open Source Software*, 6(68), 3376. <https://doi.org/10.21105/joss.03376>
- Diallo, M. S. (2022). `Samplics`: A comprehensive library for survey sampling in Python. *The Survey Statistician*, 86, 32–43.
- Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020).

- 191 Array programming with NumPy. *Nature*, 585, 357–362. [https://doi.org/10.1038/s41586-](https://doi.org/10.1038/s41586-020-2649-2)
192 [020-2649-2](https://doi.org/10.1038/s41586-020-2649-2)
- 193 Kish, L. (1965). *Survey Sampling*. John Wiley & Sons. ISBN: 978-0-471-10949-5
- 194 Lohr, S. L. (2021). *Sampling: Design and Analysis* (3rd ed.). Chapman; Hall/CRC. <https://doi.org/10.1201/9780429298899>
195
- 196 Lumley, T. (2004). Analysis of complex survey samples. *Journal of Statistical Software*, 9(8),
197 1–19. <https://doi.org/10.18637/jss.v009.i08>
- 198 Lumley, T. (2010). *Complex surveys: A guide to analysis using R*. John Wiley & Sons.
- 199 Polars contributors. (2025). *Polars: A fast multi-threaded, hybrid-out-of-core DataFrame*
200 *library*. <https://polars.rs>
- 201 Rao, J. N. K., & Scott, A. J. (1981). On the approximation of distributions of sample
202 survey statistics. *The Annals of Statistics*, 9(4), 727–748. [https://doi.org/10.1214/aos/](https://doi.org/10.1214/aos/1176345514)
203 [1176345514](https://doi.org/10.1214/aos/1176345514)
- 204 Rao, J. N. K., & Scott, A. J. (1984). On chi-squared tests for multiway contingency tables
205 with cell proportions estimated from survey data. *The Annals of Statistics*, 12(1), 46–60.
206 <https://doi.org/10.1214/aos/1176346391>
- 207 Särndal, C.-E., Swensson, B., & Wretman, J. (1992). *Model assisted survey sampling*.
208 Springer-Verlag.
- 209 Solatorio, A. V., & Dupriez, O. (2023). REaLTabFormer: Generating realistic relational and
210 tabular data using transformers. *Proceedings of the 2023 AAAI Workshop on AI for Social*
211 *Good*. <https://arxiv.org/abs/2302.02041>
- 212 Valliant, R., & Dever, J. A. (2018). *Survey Weights: A Step-by-Step Guide to Calculation*.
213 Stata Press. ISBN: 9781597182607
- 214 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
215 Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M.,
216 Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E.,
217 ... Mulbregt, P. van. (2020). SciPy 1.0: Fundamental algorithms for scientific computing
218 in Python. *Nature Methods*, 17, 261–272. <https://doi.org/10.1038/s41592-020-0772-5>
- 219 Wolter, K. M. (2007). *Introduction to Variance Estimation* (2nd ed.). Springer.
220 ISBN: 9780387350998
- 221 World Bank. (2023a). *Synthetic data for an imaginary country, full population, 2023*. World
222 Bank, Development Data Group. <https://doi.org/10.48529/78M1-AE09>
- 223 World Bank. (2023b). *Synthetic data for an imaginary country, sample, 2023*. World Bank,
224 Development Data Group. <https://doi.org/10.48529/MC1F-QH23>